# Chapter 5

## Large, Fast, and Cheap: Exploiting Memory Hierarchy

## Exercises

Thanks to Morgan Kaufmann and authors for the slides. Some slides are modified/added as per the need.

# Problem-1

## Mapping an Address to a Multiword Cache Block

### Example

Consider a cache with 64 blocks and a block size of 16 bytes. To what block number does byte address 1200 map?

Thus, with 16 bytes per block, byte address 1200 is block address

$$\left\lfloor \frac{1200}{16} \right\rfloor = 75$$

which maps to cache block number (75 modulo 64)=11. In fact, this block maps all addresses between 1200 and 1215.

MORGAN KAUFMANN

# Problem-1

## Mapping an Address to a Multiword Cache Block

### Example

Consider a cache with 64 blocks and a block size of 16 bytes. To what block number does byte address 1200 map?

1200 = 0x4B0 = 0x01**00 1011** 0000

Offset = 4-bits

Index = 6-bits = 001011= $(11)_{ten}$

# Problem-1

## Bits in a Cache

### Example

How many total bits are required for a direct-mapped cache with 16 KiB of data and four-word blocks, assuming a 64-bit address?

### Answer

We know that 16 KiB is 4096 ($2^{12}$) words. With a block size of four words ($2^2$), there are 1024 ($2^{10}$) blocks. Each block has 4 ×32 or 128 bits of data plus a tag, which is 64 −10 −2 −2 bits, plus a valid bit. Thus, the complete cache size is

$$2^{10} \times (4 \times 32 + (64 - 10 - 2 - 2) + 1) = 2^{10} \times 179 = 179 \text{ Kibibits}$$

or 22.4 KiB for a 16 KiB cache. For this cache, the total number of bits in the cache is about 1.4 times as many as needed just for the storage of the data.

# Problem-2

Assume memory is byte addressable and words are 64 bits, unless specified otherwise.

5.1 In this exercise we look at memory locality properties of matrix computation. The following code is written in C, where elements within the same row are stored contiguously. Assume each word is a 64-bit integer.

```
for (I=0; I<8; I++)
 for (J=0; J<8000; J++)
  A[I][J]=B[I][0]+A[J][I];
```

5.1.1 [5] <§5.1> How many 64-bit integers can be stored in a 16-byte cache block?

5.1.2 [5] <§5.1> Which variable references exhibit temporal locality?

5.1.3 [5] <§5.1> Which variable references exhibit spatial locality?

# Problem-2

The same computation can also be written below in Matlab, which differs from C in that it stores matrix elements within the same column contiguously in memory.

```
for I=1:8
 for J=1:8000
  A(I,J)=B(I,0)+A(J,I);
 end
end
```

# Problem-3

5.3 By convention, a cache is named according to the amount of data it contains (i.e., a 4 KiB cache can hold 4 KiB of data); however, caches also require SRAM to store metadata such as tags and valid bits. For this exercise, you will examine how a cache's configuration affects the total amount of SRAM needed to implement it as well as the performance of the cache. For all parts, assume that the caches are byte addressable, and that addresses and words are 64 bits.

5.3.1 [10] <§5.3> Calculate the total number of bits required to implement a 32 KiB cache with two-word blocks.

5.3.2 [10] <§5.3> Calculate the total number of bits required to implement a 64 KiB cache with 16-word blocks. How much

5.3.3 [5] <§5.3> Explain why this 64 KiB cache, despite its larger data size, might provide slower performance than the first cache.

# Problem-4

5.7 Consider the following program and cache behaviors.

| Data Reads per 1000 Instructions | Data Writes per 1000 Instructions | Instruction Cache Miss Rate | Data Cache Miss Rate | Block Size (bytes) |
|---|---|---|---|---|
| 250 | 100 | 0.30% | 2% | 64 |

5.7.1 [10] <§§5.3, 5.8> Suppose a CPU with a write-through, write-allocate cache achieves a CPI of 2. What are the read and write bandwidths (measured by bytes per cycle) between RAM and the cache? (Assume each miss generates a request for one block.)

5.7.2 [10] <§§5.3, 5.8> For a write-back, write-allocate cache, assuming 30% of replaced data cache blocks are dirty, what are the read and write bandwidths needed for a CPI of 2?

# Problem-5

Consider a 2-way set-associative cache of 512 Bytes size. The cache block/line size is 32 Bytes. The main memory address is 16-bits long. LRU replacement policy is used. Writes are handled through "write-back with allocate" policy. A processor uses separate instances of this cache for its I-cache and D-cache. **[3+5+2]**

a. Find the number of hits and misses observed in I-cache when executing the following RV64I code. The instruction addresses are specified at the beginning of each line. # refers to a comment.
b. Find the number of hits and misses observed in D-cache when executing the same code.
c. If we add a next-line prefetcher to the above D-cache, find the number of hits and misses, and also find accuracy of the prefetcher.

```
# x12 = 0x0100, x13 = 0x0200, x14 = 0x0380
0000:          addi x10, x0, 15
0004:   L1:    beq x10, x0, Exit
0008:          lw x5, 0(x12)
000C:          ld x6, 0(x13)
0010:          sw x7, 0(x14)
0014:          addi x10, x10, -1
0018:          addi x13, x13, 8
001C:          addi x14, x14, 4
0020:          jal x0, L1
0024:   Exit: nop
```